Suffix Trees

→ App : enumerate all substrings of S satisfying the property...

→ String of length n has n^2 different substrings

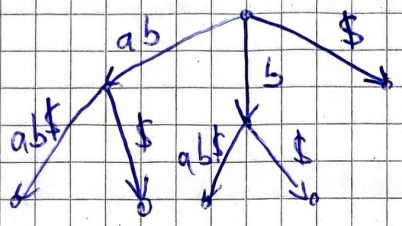
→ implicit representation → suffix tree

→ each substring of S is a prefix of a suffix of S
  ↳ n suffixes

→ unique character $ to the end of S, so that no suffix is a prefix of another suffix

→ Idea : share common prefix between the suffixes

eg : S = abab$

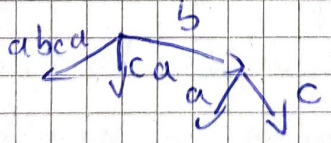


A^+-tree :

T is a finite rooted tree with properties :

1) edges are labeled by non-empty strings over alphabet A
2) ∀ node α in T and ∀ $a \in A$, there is only one a-edge $\alpha \xrightarrow{av} \beta$ for some string v and some node β

eg :



Suffix Trees

Def:

→ T a A^+-Tree

→ consists of different types of nodes and edges:

A node in T is **branching** if it has at least two outgoing edges

→ A **leaf** in T is a node in T with no outgoing edges.

→ An **internal node** in T is either the root or a node with at least one outgoing edge

→ An edge leading to an internal node is an **internal edge**

→ An edge to a leaf is a **leaf edge**

Def:

→ path(α) denotes the concatenation of the edge labels on the path from the root of T to the node α

→ paths are unique since edges are unique

→ Denote α by $\overline{w}$, iff path(α) = w

→ $\overline{\varepsilon}$ is the root node, ε is empty string

→ $\forall \overline{w}$ in T, $|w|$ is the depth of $\overline{w}$

Def:

→ a string w occurs in T if T contains a node $\overline{wu}$, for some string u

↔ if we can extend a string w by some string u and $\overline{wu}$ is a node in the A^+-tree, w occurs in the tree

↔ if there is a path for w in T

→ if w occurs in T, we say T represents w

→ occ(T): set of strings occurring in T

→ A^+-tree is compact, if $\forall$ node is either the root, a leaf or a branching node

Suffix Tree

∀ A^+-tree can be transformed into a compact A^+-tree.

Transformation: to compact tree

for each β-node, which only have one outgoing edge:
replace the edges $\alpha \xrightarrow{u} \beta$ and $\beta \xrightarrow{v} \gamma$ by $\alpha \xrightarrow{uv} \gamma$,
delete β

Suffix Tree:

Suffix Tree for $S \in A^+$, denoted by $ST(S)$ is the compact A^+-tree, s.t.

$$occ(T) = \{w \in A^* \mid w \text{ is a substring of } S\}$$

Lemma:

Suppose T is the suffix tree for S. Then:
if $\overline{w}$ is a leaf in T, then w is a suffix of S
// the reverse does not hold!

Def:

A suffix of S is nested, if it is a proper prefix of another suffix of S.

Lemma:

$\overline{w}$ is a leaf in T, iff w is a suffix of S and w is not nested

with a sentinel $ at the end, there are no nested suffixes and the one-to-one ≙ correspondence leaves ↔ suffix are easier!

Lemma:

For the suffix tree $ST(S\$)$ holds:
$\overline{w}$ is a leaf in $ST(S\$)$, iff w is a suffix of $S\$$

GIK

Role of the Sentinel Char

Def:

A substring w of S is right-branching, iff there are different characters a and b s.t. wa and wb are substrings of S

Lemma:

w is a branching node in T, iff w is a right-branching substring of S

# of leaves in suffix tree is bounded by $\mathcal{O}(n)$, $|S| = n$

Size of the suffix tree # of leaves bounded by n

# of internal nodes $\leq n-1$

balanced → compact suffix tree, each internal node is branching or it is the root

binary tree // → # of internal nodes is maximized, if each internal node has 2 outgoing edges

→ For each pair of nodes there is one internal node.

→ # of internal nodes become smaller by a factor of 2 with each of $\log_2 n$ levels towards the root.

↳ $\sum_{i=1}^{\log_2 n} \frac{n}{2^i} = n \cdot \sum_{i=1}^{\log_2 n} \frac{1}{2^i} \leq n \cdot \underbrace{\sum_{i=1}^{n} \frac{1}{2^i}}_{<1} \leq n-1$

↳ internal nodes $\leq n-1$

→ $\forall$ node {root node} there is exactly one edge leading to the node!

→ each edge points to one node → # edges $= n$ = # nodes

→ # nodes $\leq 2n-1$ ($\leq n$ leaves & $\leq n-1$ internal nodes)

→ each node in constant space

→ each edge is labeled by $S[i..j]$ of S → save (i,j) pair to refer to $S[i..j]$

→ edge can be stored in constant space

→ $2n-1$ nodes, $2n-2$ edges ⇒ Space $\mathcal{O}(n)$

GIK

Write Only Top Down Suffix Tree Construction

Simple Tree Construction Method

→ Input sequence ends with sentinel $ // Assumption

→ Write-Only-Top-Down Alg: recursive structure of a suffix tree

→ Idea: For each branching node $\bar{w}$ the subtree below $\bar{w}$ is determined by the set of all suffixes of S$ that have w as prefix

Def:

Set $R(\bar{w})$ of remaining suffix:

$R(\bar{w}) = \{S_i \mid wS_i \text{ is a suffix of } S\$\}$

1) At first $R(\bar{w})$ is divided into groups according to the first character of each suffix:

$\forall c \in A$:

$group(\bar{w}, c) = \{w \mid cw \in R(\bar{w})\}$

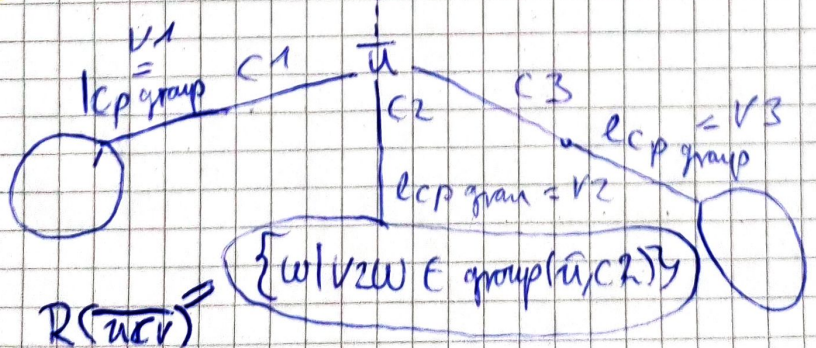
↪ c-group of $R(\bar{w})$

Case 1: If $c \in A$, $group(\bar{w}, c)$ only contains one string w, then create a leaf edge labeled cw outgoing from $\bar{w}$

Case 2: $group(\bar{w}, c)$ contains at least two strings. Then create an edge labeled cv leading to a branching node $\overline{wcv}$, where $v = lcp$ of all strings in $group(\bar{w}, c)$; the child $\overline{wcv}$ can be enumerated from the set

$R(\overline{wcv}) = \{w \mid vw \in group(\bar{w}, c)\}$

eg:

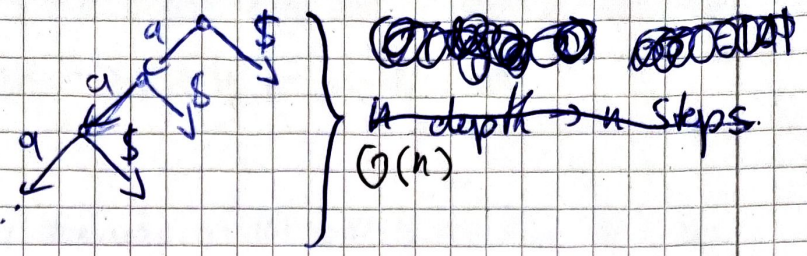


GIK

WOTD - Suffix Tree Construction

→ Running Time $O(n^2)$

eg: $S = a^n\$$



→ the suffix for $S\$$ is a binary tree with exactly one branching node of depth i, $\forall i \in [0, n-1]$

→ exactly $n-i$ suffixes are considered, to construct branching node of depth i:

$$\sum_{i=0}^{n-1} n-i = n^2 - \sum_{i=0}^{n-1} i = n^2 - \frac{n(n-1)}{2} = \frac{n^2+n}{2} \in O(n^2)$$

expected:

→ maximal depth of the branching nodes is much smaller than $n-1$, $\rightarrow O(\log_\sigma n)$, $\sigma = |A|$

↳ length of the deepest path $O(\log_\sigma n)$ // suffix along leaf edges are not read

→ expected running time $O(n \log_\sigma n)$

GIK

WOTD-Alg

Easy to parallelize:

the subtrees of the suffix tree are constructed independently from each other → easily to parallelized

Locality of data:

→ subtrees only depends on the set of remaining suffixes

→ data required to construct subtrees very small

↳ often fits # into cache (access time much smaller compared to RAM)

→ locality behaviour is very good

→ wotd-alg, fast in practice since cache access is much faster than the access to main memory

Lazy Construction:

→ many apps: traverse top-down from root down to the branching nodes until a leaf is reached

→ wotd constructs the paths top-down

→ only construct subtrees when needed for first time

↳ Lazy Construction

→ very fast

→ idea: store for an unevaluated branching node the set of suffixes it represents

GIK

Navigating in suffix trees

Location:

Let T be a compact $\mathcal{A}^+$-tree and $s \in occ(T)$.
location of s in T, denoted by $loc_T(s)$ is defined:

→ If $\bar{s}$ is a branching node, then $loc_T(s) = \bar{s}$

→ If $\bar{s}$ is a leaf node, then there is a leaf edge $\bar{u} \xrightarrow{v} s$ in T and $loc_T(s) = (\bar{u}, v, \varepsilon, \bar{s})$

→ If there is no node $\bar{s}$ in T, then there is an edge $\bar{u} \xrightarrow{vw} \overline{uvw}$ in T s.t. $s = uv$, $v \neq \varepsilon$, $w \neq \varepsilon$ and $loc_T(\bar{u}, v, w, \overline{uvw})$

If location is a node → node location else edge location.

Def: Traversal char by char

For any $\mathcal{A}^+$-tree T, any sequence s occurring in T and any $b \in \mathcal{A}$ we define the following operation:

$$follows(T, loc_T(s), b) = \begin{cases} loc_T(sb) & \text{, if } sb \text{ occurs in } T \\ / & \text{, else} \end{cases}$$

Def:

$\forall \mathcal{A}^+$-tree T, $\forall$ seq's s and w s.t. sw occurs in T:

$$getloc(loc_T(s), w) = loc_T(sw)$$

Navigating in Suffix Trees

Def:

$\forall A^+$-trees T, any seq^{ce} s occurring in T, $a \in A$ and $y \in A^*$ we denote the result of an insertion of the sequence say in T:

$$T[loc_T(s) \leftarrow ay]$$

Delivers a pair $(T', \bar{z})$:

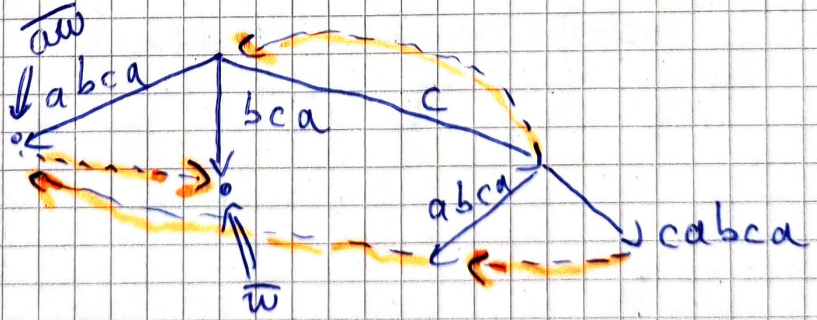
→ If $loc_T(s) = \bar{s}$, then T' is obtained from T by adding a leaf edge $\bar{s} \xrightarrow{ay} \overline{say}$,
and $\bar{z} = \perp$ ⇒ "z is undefined"

→ If $loc_T(s) = (\bar{u}, v, w, \overline{uvw})$, then T' is obtained from T by splitting $\bar{u} \xrightarrow{vw} \overline{uvw}$ into $\bar{u} \xrightarrow{v} \bar{s} \xrightarrow{w} \overline{uvw}$ and adding a new leaf edge $\bar{s} \xrightarrow{ay} \overline{say}$,
and $\bar{z} = \bar{s}$, $\bar{z}$ is the new inner node created by the splitting

Def:

T a compact A^+-tree and consider a node $\overline{aw}$ in T.
If $\overline{w}$ is a node in T, then the suffix link for $\overline{aw}$ is an edge from $\overline{aw}$ to $\overline{w}$

eg:



GUK

Navigating in suffix tree

Def:

$\forall A^+$-tree T, let $\mathcal{L}$ be a set of all suffix links in T. We extend suffix links to locations:

1) $linkloc_{\mathcal{L}}(\bar{s}) = \begin{cases} \bar{\varepsilon} & , \text{if } \bar{s} \to \bar{\varepsilon} \in \mathcal{L} \\ / & , \text{else} \end{cases}$

2) $linkloc_{\mathcal{L}}(\bar{u}, av, w, \overline{uavw}) = \begin{cases} loc_T(v) & , \text{if } \bar{u} = root \\ get\text{-}loc(\bar{z}, av) & , \text{elif } \bar{u} \to \bar{z} \in \mathcal{L} \\ / & , \text{else} \end{cases}$

→ observe: $get\text{-}loc(\bar{\varepsilon}, v)$

eg:

$loc_T(c)$

$linkloc_{\mathcal{L}}(root, bc, a, \overline{bca}) = \bar{c}$ // $src: bc$ $target: c$

$loc_T(bc)$

$linkloc_{\mathcal{L}}(root, abc, a, \overline{abca}) = (root, bc, a, \overline{bca})$ // $src: abc$ $target: bc$

$linkloc_{\mathcal{L}}(\bar{c}) = root$, $linkloc_{\mathcal{L}}(\bar{\varepsilon}, a, bca, \overline{cabca}) = (root, a, bca, \overline{abca})$

↳ if $\bar{c} \to root$ add $av = a$ εa // $ca - a$

Overview:

node location	: we are at a branching node / root
edge location	: we are between two nodes
follow	: traverse down from location by one symbol
get-loc	: traverse down from location by one or more symbols
$T[loc_T(s) \leftarrow eq]$	: insert suffix pair in T at $loc_T(s)$
suffix link for $\overline{aw}$	: unlabeled edge from $\overline{aw}$ to $\bar{w}$ if $\bar{w}$ exists
$linkloc_T$	: link location via suffix links

Ukkonen

Linear-time suffix tree construction of Ukkonen

→ Input string ends with sentinel $

→ Idea: construct ST by successively inserting the suffixes of S$ into an initially empty tree from longest to shortest.

$ST_0, \dots, ST_{n+1}$

ST_0 is the empty A^+-tree

→ $i \in [1, n+1]$, ST_i is the compact A^+-tree s.t.

$$occ(ST_i) = \{w \in A^* \mid w \text{ is a prefix of } x_j,\ j \in [1, i]\}$$

$$, x_j = S\$[j \dots n+1]$$

→ $ST_{n+1} = ST$

→ ST_{i-1} and ST_i differ by the added suffix $x_i = S\$[i \dots n+1]$, which occur in ST_i but not in ST_{i-1}

→ Location where to insert:

→ $head_1 = \varepsilon$ and $\forall i \in [2, n+1]$ let $head_i$ be the longest prefix of x_i that is the same as x_j, $\exists j \in [1, i-1]$

→ $tail_i$ = remaining suffix of x_i, $head_i tail_i = x_i$

→ $tail_{n+1} = \$$ and $tail_i \neq \varepsilon \ \forall i \in [1, n+1]$

→ branching node $v \neq root$ in ST_i, iff $v = head_j \neq \varepsilon$, $\exists j \in [2, i]$

↳ the path leading to a branching node is a head-sequence of

for each head-sequence a branching node exists, whose path is the head-sequence

→ head is represented by a location and a tail is represented by a start position of a suffix of S$

$headloc_i = loc_{ST_{i-1}}(head_i)$

$tailptr_i \in [1, n+1]$ s.t. $tail_i = S[tailptr_i \dots n]\$$, $\forall i \in [1, n+1]$

Ukk

Linear time suffix tree construction of McC

→ with each ST_i, $i \in [1, n]$, also the suffix links for each node $\overline{head_j} \neq root$, $j \in [2, i-1]$ is constructed

↳ only suffix links for branching nodes are constructed and the only branching node in ST_i whose suffix link is possibly not constructed, is node $\overline{head_i}$

→ ST_1 is computed A^+ tree with one edge $root \xrightarrow{S\$} S\$$ // $O(1)$

→ to compute ST_i from ST_{i-1} it is crucial to compute headloc$_i$ / tailptr$_i$

↳ then only a single tree insertion operation needs to be done:

$(ST_i, \bar{r}) = ST_{i-1}[headloc_i \leftarrow tail_i]$

, $\bar{r} = \perp$ or $\bar{r}$ is the new branching node for which no suffix link is constructed yet

Def:

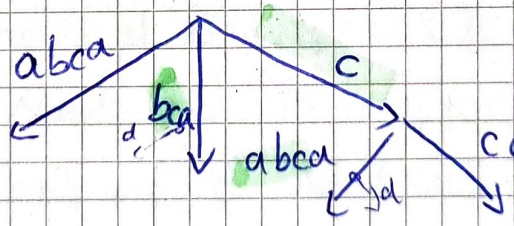
Let T be an A^+-tree. $\forall S \in occ(T)$, $\forall$ strings w, and $\forall j \in [1, |w|]$, the function scanprefix is specified:

scanprefix$(loc_T(s), w, j) = (loc_T(su), j + |a|)$

↳ u is longest prefix of $w[j..|w|]$ s.t. $su \in occ(T)$

↳ find loc where to insert an edge.

eg:



scanprefix$(loc_T(cab), \underbrace{cd}_{w}, \underbrace{1}_{j}) = (loc_T(cabc), 2)$

scanprefix$(loc_T(bca), \underbrace{d}_{w}, \underbrace{1}_{j}) = (loc_T(\underbrace{bca}_{su}), \underbrace{1}_{j+|w|})$

u is longest prefix of $w[j..|w|] \rightarrow su \in occ(T)$

Mc Creights suffix tree Construction

In: $S\$$
Out: $ST(S\$)$ and set of all suffix links for branching nodes

Let T be the Λ^+-tree with a single edge root $\xrightarrow{S\$}$ $\overline{S\$}$ // ST_1

$\mathcal{L} \leftarrow \emptyset$ // suffix links for branchy in ST_{i-2}
$loc \leftarrow root$ // $headloc_i$, $i \geq 1$
$tailptr \leftarrow 1$ // $tailptr_i$, $i \geq 1$
$\bar{z} \leftarrow \perp$ // $\perp$ or br. node created in last step

for $i \leftarrow 2$ up to $n+1$ do // $ST_2, \ldots, ST_{n+1}$
  if $loc \neq root$ then // $head_{i-1} = \varepsilon$
    $(loc, tailptr) \leftarrow scan_{root}prefix(loc, S\$, tailptr+1)$ // skip $S[tailptr]$ (is previously processed)
    $(T, \bar{z}) \leftarrow T[loc \leftarrow S[tailptr \ldots n+1]]$
  else // $head_{i-1} \neq \varepsilon$
    $loc' \leftarrow link loc_{\mathcal{L}}(loc)$ // skip $head_{i-1}[0]$
    $(loc, tailptr) \leftarrow scan prefix(loc', S\$, tailptr)$ // $headloc_i$ / $tailptr_i$
    $(T, \bar{r}) \leftarrow T[loc \leftarrow S[tailptr \ldots n+1]]$
    if loc' is a node then
      $\mathcal{L} \leftarrow \mathcal{L} \cup \{\bar{z} \rightarrow loc'\}$ // create suffix link
      $\bar{z} \leftarrow \perp$ // no br. node with undefined s.l.
    else
      $\mathcal{L} \leftarrow \mathcal{L} \cup \{\bar{z} \rightarrow \bar{r}\}$ // create suffix link
      $\bar{z} \leftarrow \bar{r}$ // new node $\bar{r}$ with undefined s.l.
    endif
  endif
endfor

new leaf

$head_{i-1} = a\, head_i$
↳ link to $head_i$

→ loc is the location of $head_{i-1}$ in ST_{i-2}
→ if not root → represents aw, $a \in A$, $w \in A^*$
→ aw is a non-empty prefix of $S[i-1 \ldots n]$, $a = S[i-1]$, w is prefix of $S[i \ldots n]$
→ $head_i$ is longest prefix of $S[i \ldots n]$, w must be a prefix of $head_i$
↳ instead of determining the location of $head_i$ by first scanning w from the root, one more directly determines the location loc' of w by applying the $link loc_{\mathcal{L}}$-Function

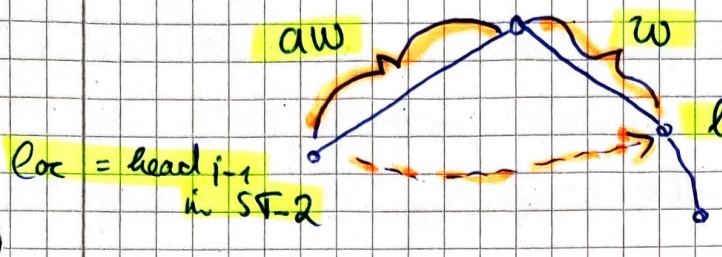
$head_{i-1} = a\, head_i$
($head_i = aw$ → $wy \rightarrow y$)

// suffix link for $\overline{aw}$ is an edge $\overline{aw}$ to $\overline{w}$

GIK

McCreight

→ if the branching node for $head_{i-1}$ was not constructed in step $i-1$, loc is a branching node with a suffix link, which is followed to the branching node for w.
↳ this branching node already exists and is loc'



→ if the branching node for $head_{i-1}$ was constructed in step $i-1$, loc is an edge location in ST_{i-2}
↳ we follow the suffix link of the parent node of the branch location for $head_{i-1}$ to reach a node which is on the path from the root for the location of w.

→ suffix links allow finding leaves in linear time.

GIK

McCreight

-> Insertion -> constant time $O(1)$

-> scanprefix is called $n-2$ times

-> each call time $\sim$ to the length of the scanned prefix

=> as tail ptr never decreases, total runn. time of scanprefix $O(n)$

-> filling a link in $link.loc_i$ -> $O(1)$

-> computation of $loc_T(v)$ and $getLoc(\bar{z}, av)$ in $link.loc_i$ introduces seq's, which already occurr in current st^i-tree

=> traversal only requires to look at the first char of each edge

=> each edge can be traversed in $O(1)$

=> total runn time of $link.loc_i$ is in $O(n)$

Theorem:

Let $S \in A^n$. Alg mcc constructs $ST(S\$)$ in $O(n)$ Time and additionally delivers the set of suffix links $\forall$ branching nodes of $ST(S\$)$

@IK

Searching for Exact Patterns

$P \in A^*$, which does not contain \$.

A) Verify whether P is a substring of S.

↳ follow the path from the root directed by the characters of P

↳ If at some point one cannot proceed with the next char in P, then P does not occur in the suffix tree and hence it is not a substring of S

else P is substring of S.

↳ Runtime $O(|P|)$ → does not depend on length of S

↳ But, worst time in $ST(S\$)$

B) Find Positions where P occurs in S

→ annotate the leaves of the suffix tree

→ $v\$$ has a leaf number, that is, the position where the corresponding suffix $v\$$ starts.

→ while scanning P we reach $\bar{w}$ ⇒ Enumerate all leaf numbers of the leaves in the subtree below $\bar{w}$

↳ these are P in S

Variation:

search for the largest prefix of P, say P', that is a substring of S. → $O(|P'|)$ time

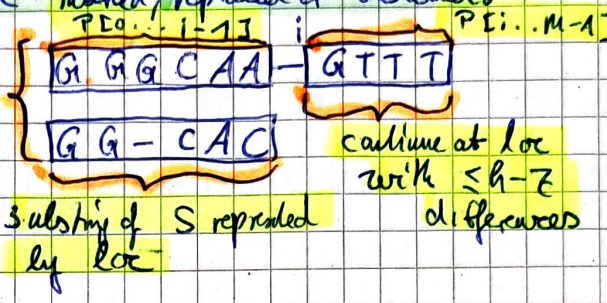
Approximate Matching of Patterns against a suffix tree

→ Pattern matching problem with allowing a maximum unit edit distance k when matching P against the suffix tree $ST(S\$)$ of $S\$$

→ simple method: for small k

↳ extend a partial approximate match character by character, s.t. the characters are inserted, replaced or deleted.

$edist = z \leq k$



→ At any stage of the extension we need to keep track of the location at which we are in the suffix tree

→ Each call of the form approxmatch(P, k, loc, i, z) means that one has reached the location loc after matching the prefix P[0..i-1] with z differences

↳ represent substring w of S
↳ edist(w, P[0..i-1]) = z

→ if $z \leq k$, continue at loc with now at most $k - z$ differences

GIV-

Approximate Matching of Patterns against a suffix tree

Recursive function for computing the set of positions j in sequence S s.t. the unit edist of P and of a prefix of $S[j..n]$ is at most k.

Call approxmatch ($P, k,$ root, $1, 0$)

In: $ST(S\$)$ with annotated leafs

Out: set of positions in S where an approx. match begins

```
approxmatch (P, k, loc, i, z)
    if z > k then
        return ∅
    endif
    if i = |P| then                    // P[0..i-1] = P matches
        return {j | j is annotation of some leaf in subtree below loc}
    endif
    A ← approxmatch (P, k, loc, i+1, z+1)        // Deletion
    for all b ∈ A do
        if loc_b ← follow (ST(S$), loc, b) is defined then
            A ← A ∪ approxmatch (P, k, loc_b, i, z+1)        // Insertion
            if b = P[i] then
                A ← A ∪ approxmatch (P, k, loc_b, i+1, z)        // Match
            else
                A ← A ∪ approxmatch (P, k, loc_b, i+1, z+1)        // mismatch
            endif
        endif
    endfor
    return A
```

→ Running Time $\exp(k)$

GLK

Approximate Matching of Pattern against a suffix tree

→ Reduce Search Space

→ Idea: Discard matching a suffix $P[i..m]$ of P, if the remaining # of $k-z$ possible differences does not suffice to complete the match

→ compute table $M: [0, m-1] \to \mathbb{N}$, s.t.
$M(q)$ = maximal matches distance of $P[q..m-1]$ and S

→ Can be computed from the right-to-left partition
$(w_1, c_1, w_2, \dots, w_r, c_r, w_{r+1})$ of P w.r.t. S
↳ Remember: → $P = w_1 c_1 \dots c_r w_{r+1}$)
→ $\forall q \in [1, r+1]$, w_q is substring of S
→ $\forall q \in [1, r]$, $c_q w_{q+1}$ is not substring of S

→ c_i marked character

→ maximal matches distance of $P[q..m-1]$ and S
= # of marked characters in $P[q..m-1]$ which separates matches from substrings of S. // position where difference must occur

→ mmdist is a lower bound for the unit edist
↳ if it is larger than some value $k-z$, then the unit edist is also larger than $k-z$

→ Table M is used in function approxmatch by replacing $z > k$ by: $z > k$ or $(i \leq m-1$ and $k-z < M(i))$

→ Compute M via the right-to-left partition requires to compute the ST of the reverse of seq S
↳ $\mathcal{O}(n)$ time and space

GK

Minimal unique substrings

App: Primer Design

→ Enumerating all substrings x of some string S, s.t.:

1) x occurs exactly once in S (uniqueness)
2) all proper prefixes of x occur at least twice in S (minimality)
3) x is of length at least ℓ, for some fixed ℓ

eg: $S = abab$, $\ell = 2$ ⇒ aba & ba

⇒ bab not since prefix ba is already unique

→ the ST(S$) properties:

1) If a string w occurs at least twice in S, there are at least two suffixes in S$ of which w is a proper prefix.
In ST(S$), w corresponds to a path ending with an edge to a branching node

2) If a string w occurs only once in S, there is only one suffix in S$ of which w is a prefix.
In ST(S$), w corresponds to a path ending with an edge to a leaf
↳ find the unique strings by looking at the paths ending on the edges to a leaf // only enumerate ending leaf edges

→ suppose $\overline{w} \xrightarrow{av} \overline{wav}$ is an edge from $\overline{w}$ leading to a leaf $\overline{wav}$, $a \neq \$$

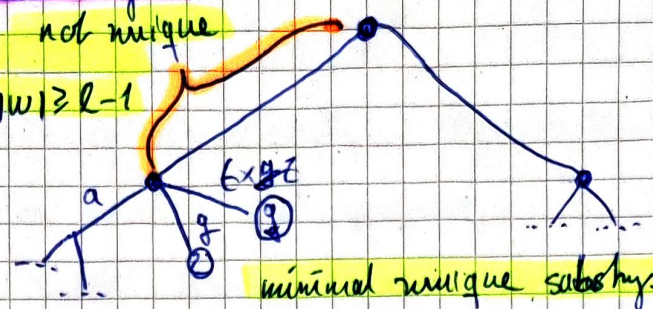
→ Then wa occurs exactly once in S = it's unique

⇒ Only check if the length of wa is at least ℓ

$|w| \geq \ell - 1$

Minimal Unique Substrings

graphical:



Alg (Minimal Unique Substrg):

→ enumerate all branching nodes $\overline{w}$ of ST(S$) s.t. $|w| \geq k-1$

→ for each such node enumerate all edges $\overline{w} \xrightarrow{av} \overline{wav}$ leading to a leaf $\overline{wav}$, $a \neq \$$. Output wa

Running Time:

→ linear in the # nodes and # edges in ST(S$), since we have to visit each of these only once and for each we do a constant amount of work

→ ST(S$) can be constructed in linear time

↳ there are $O(n)$ nodes and edges

→ optimal since running time is linear in the size of input

GWK

Maximal Unique Matches

-> ~~done~~ use smethy else than edits

-> Idea: Similarity often results in long identical substrings which occur in both genomes

<-> those are called MUMs (maximal unique matches) are almost surely part of any good alignment of the two genomes

-> Find MUMs -> the subjects to a chaining method and a resulting chain is taken as the anchor of an alignment

-> Then remaining parts are aligned using traditional DP-Methods

Maximal Unique Math:

two sequences $S, S' \in A^*$ (the genomes) and a positive integer l. Maximal-Unique-Matches problem (MUM-problem) is to find all MUMs, i.e. all seq's u with:

-> $|u| \geq l$

-> u occurs exactly once in S and it occurs exactly once in S' (uniqueness)

-> $\forall a$ neither ua nor au occurs both in S and in S' (maximality)

eg:

$S = cctctcgt$

$S' = ctgtcgt$, $l = 2$

$$A = \begin{matrix} cct-tcgt \\ -ctgtcgt \end{matrix} \rightarrow MUMs = \{ ct, tcgt \}$$

GAK

Maximal Unique Matches

→ To compute MUMs, compute $ST(S\#S'\$)$ (also suff. tree) $= ST(X)$

→ A MUM u must occur exactly twice in X, once left of # (in S) and once right of # (in S')

→ u corresponds to a path in the suffix tree $ST(X)$ ending with an edge to a branching node.

→ u is right-maximal (i.e. ua not in S & S'), u must occur with two different right contexts

↳ u must even correspond to a branching node.

↳ For each MUM u there exists a branching node $\bar{u}$ in $ST(X)$

→ Since u occurs twice ⇒ exactly two leaves in the subtree below $\bar{u}$

→ Subtree contains no branching nodes

→ Given a branching node $\bar{u}$ the existence of exactly two such leaf edges can easily be checked in constant time

↳ one edge must lead to a leaf, say $\bar{v}$, that corresponds to a suffix starting in S, and the other edge must lead to a leaf, say $\bar{w}$, that corresponds to a suffix starting in S'

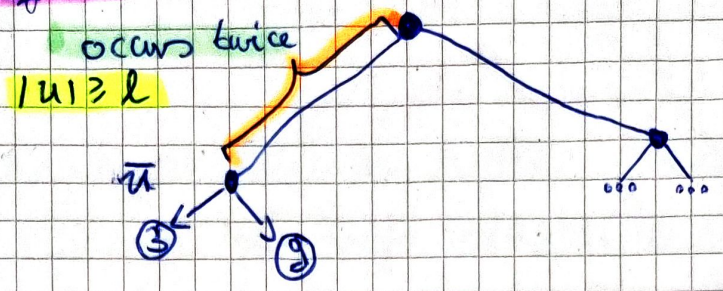
→ Verify left-maximality, i.e. to check that there is not character a s.t. au occurs in both S and S'

↳ Suppose that leaf $\bar{v}$ has leaf number i and leaf $\bar{w}$ has leaf number j, s.t. $i < j$

↳ u is left-maximal, iff $i = 0$ or $X[i-1] \neq X[j-1]$

↳ constant time

Maximal Unique Matches



$3 < pos_\# < 9$, $X[3-1] \neq X[9-1]$

$\Rightarrow$ MUM u at position 3 in S and $9 - pos_\#$ in S'

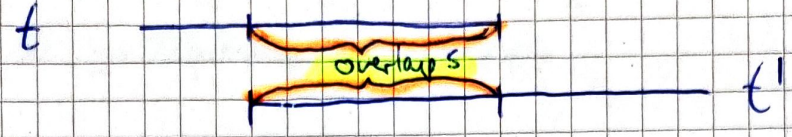
→ $n = |S| + |S'|$

→ Check $\mathcal{O}(n)$ branching nodes in ST ($X\$$) and for each branching node we have to perform different tests in constant time → Runtime $\mathcal{O}(n)$

All-pairs suffix-prefix matching

Def

For two sequences t and t', any suffix of t that matches a prefix of t' is a suffix-prefix match of t and t'



Def:

Given a set S of sequences, the all-pair suffix-prefix matching problem (APSMP) for S comes in two variations

→ APSMP-problem with minimum length ℓ is to compute all triples (t, t', r) where $t, t' \in S$ and there is an SPM of t and t' of length $r \geq \ell$. $\ell > 0$

→ the maximum APSPM-problem is to compute all triples (t, t', r) where $t, t' \in S$ and r is the length of the longest SPM of t and t'

GIK

All-pairs suffix-prefix problem with minimum length

Let $S = \{s_0, \dots, s_{k-1}\}$. To solve APSPM-problem with minimum length, we construct ST(T):

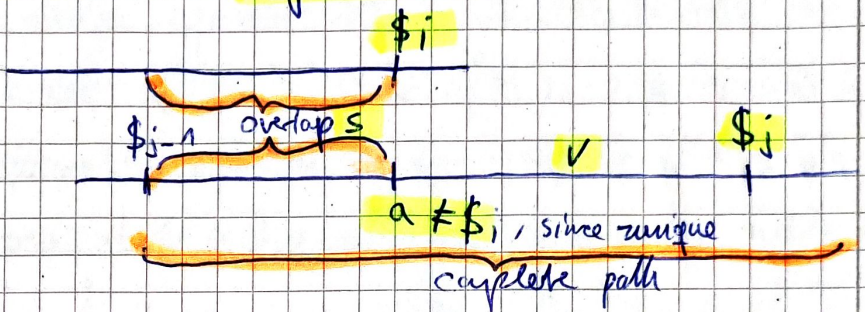
$$T = s_0 \$_0 s_1 \$_1 \dots \$_{k-2} s_{k-1} \$_{k-1}$$

with unique seperators $\$_0, \dots, \$_{k-1}$.

→ Consider seq's s_i and s_j, $0 \le i, j \le k-1$, $i \neq j$

→ suffix s of s_i matches prefix of s_j (i.e. s_j starts with s)

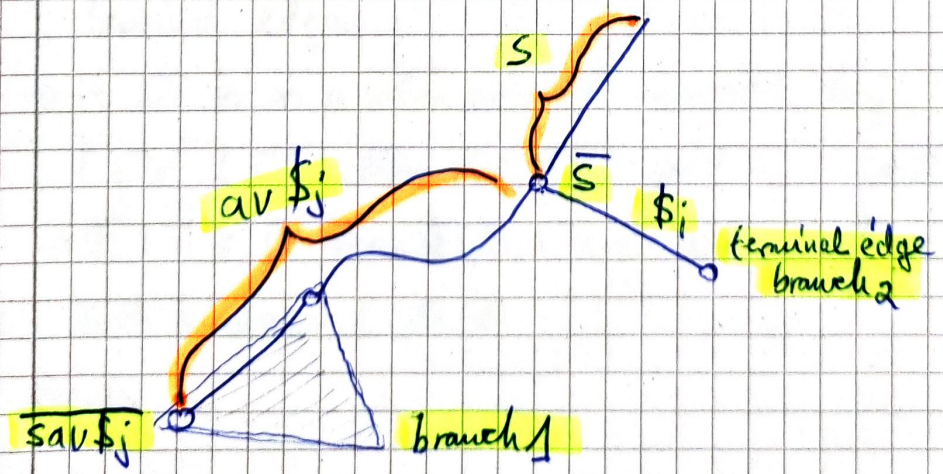
↳ $s_i = us$ & $s_j = sav$



→ The suffix tree ST(T) will contain $s\$_i$ & $s_j\$_j = sav\$_j$

→ s is prefix of $s\$_i$ & $sav\$_j$ → s is a right-branching substring of T → s ends in branching node $\bar{s}$ in ST(T)

↳ from $\bar{s}$, one branch (branch 1) will continue $av\$_j$ and other branch (branch 2) will continue $\$_i$



GIK

All-pairs suffix-prefix matching with minimum length

→ edges with label starts $\$_i$, $i \in [0, h-1]$ are called terminal edges → leaf edges

→ Collect information about terminal edges at each branching node in the set $\alpha(\bar{s})$, i.e. $i \in \alpha(\bar{s})$ iff there is a terminal edge outgoing from the branching node $\bar{s}$ whose label begins with $\$_i$

→ the α sets are not pairwise disjoint! $\$_i$ in multiple α's

→ path starting from $\bar{s}$ along branch i:
reaches the unique separator $\$_j$ as it starts from a_{ij} and reaches the leaf $\overline{s_j \$_j}$ ..... (short $\overline{s_j \$_j}$) → spells whole seq s_j

→ Paths representing a whole sequence will be called whole paths and leaves at the end of whole paths are called whole leaves

→ $\bar{w}$ a leaf & q the start position of the suffix w of T

→ $\bar{w}$ corresponds to a whole path for s_j, iff $q = 0$ $(j = 0)$ or $T[q-1] = \$_{j-1}$, $j > 0$

⇒ An SPM can be characterized by branching nodes which are on a whole path and which have an outgoing terminal edge

→ If $\bar{s}$ is on a whole path to some whole leaf $\overline{s_j \$_j}$ and $i \in \alpha(\bar{s})$, then s is a prefix of s_j and a suffix of s_i

→ If $|s| \geq \ell$, then s is a solution to the APSPM-problem with minimum length ℓ

GIK

All-pairs suffix-prefix matching with minimum length

-> traverse suffix tree in depth first order.

Alg: Computation of all SPHs of minimum length

In: ℓ, ST(T), each node v is annotated by its depth $v.depth$ & parent $v.parent$ if exists
$s_0 \$_0 \ldots s_{k-1} \$_{k-1}$

Out: all SPHs of length at least ℓ, $\forall$ pairs of strings from S

```
root.visited ← false
W(root) ← ∅
push(tovisit, root)                    // stack of tree nodes
while tovisit is not empty do          // Keep on stack for second visit
    v ← stacktop(tovisit)
    if not v.visited then
        λ(v) ← ∅
        v.visited ← true
        for all edges v→w outgoing from v do
            if w is a branching node then
                w.visited ← false
                push(tovisit, w)
            else if w corresponds to s_j $_j for some j, 0 ≤ j ≤ k-1 then   } leaf
                W(v) ← W(v) ∪ {j}                                          } whole
            else if label of v→w starts with $_i, 0 ≤ i ≤ k-1 then
                λ(v) ← λ(v) ∪ {i}
            endif
        endfor
    else                               // v is visited // subtrees bellow are processed
        if v.depth ≥ ℓ then
            W(v.parent) ← W(v.parent) ∪ W(v)        // up propagate the set
            for i ∈ λ(v) do
                for j ∈ W(v) do
                    output(s_i, s_j, v.depth)
                endfor
            endfor
        else
            W(v) ← ∅
        endif
        Stackpop(tovisit)              // processing finished
    endif
endwhile
```

$\overline{s_j \$_j}$

W(v) = all whole leaves reached from v

λ(v) = all $\$_i$ stub edges from v

GWK

All-pairs suffix-prefix matching with unmin length

→ nodes are visited twice & edges are visited once

→ $\mathcal{O}(1)$ if whole leaf

→ Stack operations $\mathcal{O}(1)$

→ $\mathcal{W}$- and λ- sets implemented by a range of consecutive indices of two global arrays storing sequence numbers of suffixes sorted lexicographically

→ Traversal of the suffix tree and the maintenance of the λ- / $\mathcal{W}$- ~~table~~ sets takes $\mathcal{O}(n)$ time.

$n = \sum_{i=0}^{R-1} (|s_i|+1)$ is the total length of the concatenated seq's.

↳ $\mathcal{O}(n+z)$ where $z = \#$ SPMs → total running time $\mathcal{O}(n+z)$

|input| |output| ⇒ optimal!

→ size of the union of all λ- and $\mathcal{W}$-sets is limited by n and the # of these sets represented at any time is limited by the depth of the stack

↳ their size is bounded by $\mathcal{O}(n)$

→ Suffix tree requires $\mathcal{O}(n)$ space → total space $\mathcal{O}(n)$

GIK

Maximum all-pairs suffix-prefix matching

-> SPM : whole path and terminal edge

-> Find longest SPM of s_i and s_j,
combine s_j represented by a whole leaf with the deepest node $\bar{s}$ on the corresponding whole path s.t. $i \in \alpha(\bar{s})$
↳ s max prefix of s_j and suffix of s_i => longest SPM of s_j and s_i

-> DFS & $\forall i \in [0, k-1]$, keep a stack called stack$_i$ on which we push the depth of branching nodes

-> First visit of $\bar{s}$: push $\bar{s}$.depth on stack$_i$, $\forall i \in \alpha(\bar{s})$
↳ Top element of stack$_i$ stores the depth of the deepest node representing a suffix of s_i matching some substring of some sequence in S.

-> When a whole leaf representing $s_j \in S$ is reached, ~~the~~
the top element of stack$_i$ is the length of the maximum SPM of s_i and s_j

-> Second visit of $\bar{s}$: delete the top element from stack$_i$, $\forall i \in \alpha(\bar{s})$

GIK

Maximum all-pairs suffix-prefix matching

Algo: Gusfield

In: ST(T)

Out: maximal SPHs of each pair of strings in S

```
∀ i ∈ [0, k-1], let stack_i be initialized empty
root.visited ← false
push (tovisit, root)
while tovisit is not empty do
    v ← stacktop(tovisit)
    if not v.visited then
        for all edges v -> w outgoing from v do
            if v -> w is a terminal edge whose label begins with $_i then
                push (stack_i, v.depth)
            endif
        endfor
        v.visited ← true
        for all edges v -> w do
            if w is a branching node then
                w.visited ← false
                push (tovisit, w)
            elseif w corresponds to s_j, j ∈ [0, k-1] then // whole leaf
                for i ← 0 upto q-1 do
                    if stack_i not empty then
                        output (s_i, s_j, stacktop(stack_i))
                    endif
                endfor
            endif
        endfor
    else
        for all edges v -> w do
            if v -> w is a terminal edge whose label begin with $_i then
                stackpop (stack_i)
            endif
        endfor
        stackpop (tovisit)
    endif
endwhile
```

Gn1K

Maximum all-pairs suffix-prefix matching

→ # terminal edges $\leq n$, n is $|T|$

→ push and pops on stack; associated with terminal edges

↳ traversal with update of the stack takes $O(n)$ time

→ $O(k^2)$ solutions each created in $O(1)$

→ Algorithm runs in $O(k^2+n)$ time

↳ optimal → in input size → k^2 size of the output

→ ST(T) stored in $O(n)$ space

→ visit stack is bounded by # nodes of ST(T) → $O(n)$ space

→ stacks altogether contain as many elements as there are terminal edges on the current path → $O(n)$ space

↳ Total $O(n)$ space

Maximal Repeats

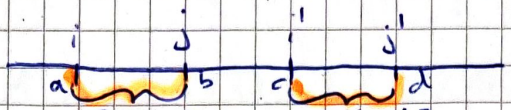
Find exact repeats. Compute all maximal repeats of a input sequence

→ $S = S[0..n-1]$

→ substring $S[i..j]$ of is repeated by pair (i,j)

→ A pair $((i,j),(i',j'))$ in S is a repeat if $i < i'$ and $S[i..j] = S[i'..j']$

↳ (i,j) is left instance of the repeat & (i',j') is right instance

$S =$ 

$S[i..j] = S[i'..j']$ ⇒ maximal $a \neq c$, $b \neq d$

Def:

A repeat is maximal if it is left maximal and right maximal.

→ repeat $((i,j),(i',j'))$ is left maximal, iff $i-1 < 0$ or $S[i-1] \neq S[i'-1]$

→ repeat $((i,j),(i',j'))$ is right maximal, iff $j'+1 > n-1$ or $S[j+1] \neq S[j'+1]$

→ all non-maximal repeats can be obtained by maximal repeats

→ Alg to compute all maximal repeats:

Phase 1: the leaves of the suffix tree are annotated

Phase 2: the repeats are output while simultaneously annotating the branching nodes

GIK

Maximal Repeats

→ We have $ST(S)$, $|S| = n$, $S \in \Delta^*$

→ Ignore leaf edges from the root → repeats of length zero

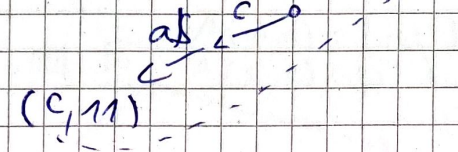
→ Phase 1:

- if $v = S[i..n]$, then the leaf $\bar{v}$ is annotated by pair (a, i), where i is the position where the suffix v starts and $a = S[i-1]$, the char left to that position
  ↳ (a, i) is the leaf annotation of v

- write $A(\bar{v}, S[i-1]) = \{i\}$ to denote the annotation.
  $A(\bar{v}, c) = \emptyset$, $\forall c \in \Delta$ different from $S[i-1]$
  ↳ or whenever there is no annotation (c, j) for some j

eg:

a g g c g c t g c g c c a $



- character a helps to decide the left-maximality of a repeat
  ↳ i position where repeat occurs

Maximal Repeats

→ Phase 2: ↳ branch node is visited after the subtree below is visited

- In a bottom-up traversal the repeats are output and the annotation for the branching nodes is computed
- Process edge $\bar{w} \xrightarrow{au} \bar{v}$:

↳ At first repeats (for $\bar{w}$) are outputted by combining the annotation already computed for node $\bar{w}$ with the complete annotation stored for $\bar{v}$ (bottom up)

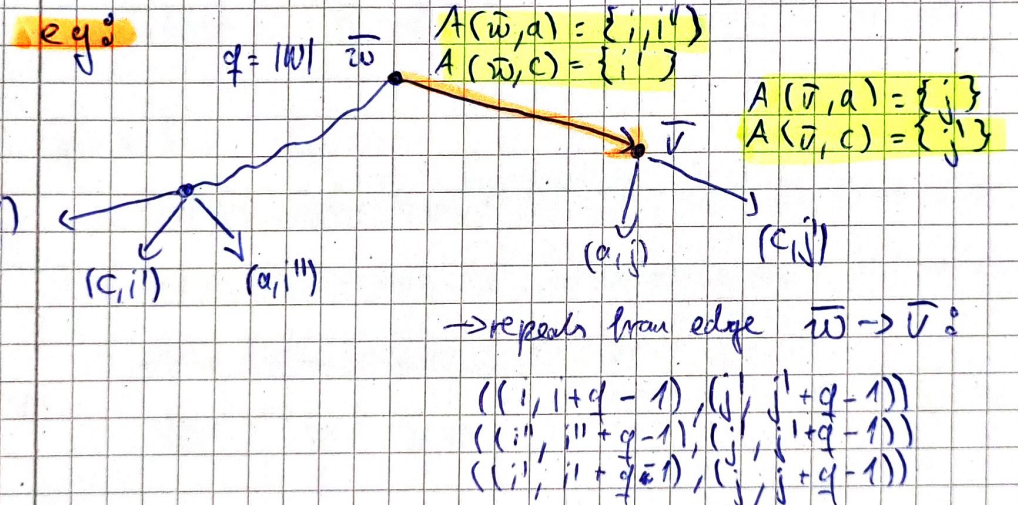
↳ We output all pairs $((i, i+q-1), (j, j+q-1))$:

→ q is the depth of node $\bar{w}$, $q = |w|$

→ $A(\bar{w}, c)$ already computed for $\bar{w}$ w.r.t. char c and $A(\bar{v}, c')$ stored for node $\bar{v}$ w.r.t. char c'

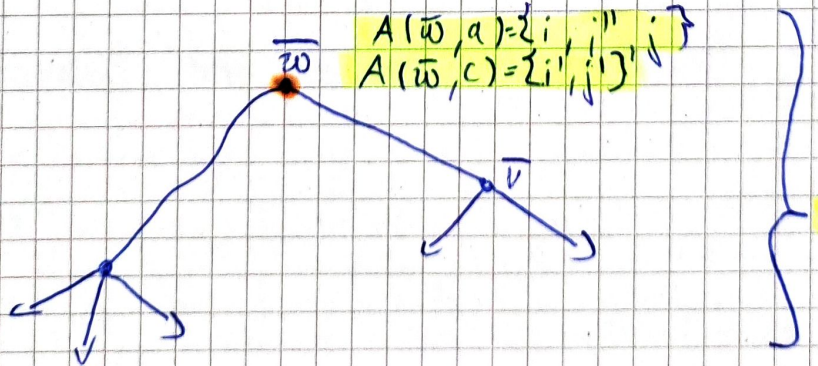
→ $i \in A(\bar{w}, c)$ and $j \in A(\bar{v}, c')$, for $c \neq c'$ ⇒ left-maximality

eg:



→ repeats from edge $\bar{w} \to \bar{v}$:

$((i, i+q-1), (j, j+q-1))$
$((i'', i''+q-1), (j, j+q-1))$
$((i', i'+q-1), (j', j'+q-1))$



different

⇒ $\bar{w} \xrightarrow{au} \bar{v}$, ↳ $c \neq a$ → w is repeated right, c & a are characters to the right → different right contexts

↳ also right-maximal

↳ visit to $\bar{w}$

GIK

Maximal Repeats

→ After processing all edges outgoing from $\bar{w}$:

This node is annotated by the set of positions where w occurs, and this set is divided into disjoint subsets $A(\bar{w}, a_1), \ldots, A(\bar{w}, a_r)$ where $A = \{a_1, \ldots, a_r\}$

Runtime:

→ bottom up traversal $O(n)$

→ Output & Combination of annotations: implemented as link lists, then the output operation can be implemented → linear in # repeats.

• combing annotations, linking lists together, can be done in linear time in # nodes visited during traversal

→ ST(S) constructed in $O(n)$

→ Total $O(n+z)$, $n = |S|$, z = # repeats

Space:

→ do not store annotations for all nodes all at once

→ as soon as a node and its parent is processed → annotation not needed anymore

→ Annotation requires $O(n)$ space

→ Alg $O(n)$ space

⇒ Optimal linear in size of input plus output

GIK

Overview of suffix-tree

Problem	Input	Method	Runtime
exact pattern match	S, P	top down trav.	$O(\|S\|+\|P\|)$
approx. patt. match	S, P, k	recursive top down trav.	$O(\|P\|^k)$
Min unique Substring	S, ℓ	enumeration of leaf edges	$O(\|S\|)$
Max unique Matches	S, S', ℓ	enumeration of branching nodes with exactly 2 leaf edges	$O(\|S\|+\|S'\|)$
min. len. APSPM	$\mathcal{S}, \ell$	depth first search, collect L/W sets	$O(n+z)$
max. APSPM	$\mathcal{S}$	DFS, terminal edges trigger stack push/pop, combine stack tops at inside leaves	$O(n+\|\mathcal{S}\|^2)$
max. exact repeats	S, ℓ	bottom-up traversal, combine annotation at by edges, inherit them	$O(\|S\|+\sigma)$

$\mathcal{S}$: set of sequences

$n = \sum_{s \in \mathcal{S}} (|s|+1)$, z = output size

GIK – ÜS

5.1

Alphabet A, String $S \in A^+$

$Absent(A,S) = \{w \in A^+ \mid w \text{ ist kein Substring von } S\}$

unendlich viele

$\rightarrow w \in Absent(A,S)$: abwesendes Wort

Mindest Länge eines abwesenden Worts:

$\mu(A,S) = \min\{|w| \mid w \in Absent(A,S)\}$

Alg: (abwesende Zeichenkette einer minimalen Länge mit $ST(S\$)^*$)

BFS

```
enum_absent(strea):
    queue_ = queue.SimpleQueue()
    queue_.put((strea.root(), []))
    min_absent = []
    size_min_absent = -1
    while not queue_.empty():
        node, strij = queue_.get()
        if not node
            continue
        for x in range(strea.alphabet_size()):
            for_b = strea.follow(node, x)

            if for_b: # match
                if -1 != size_min_absent <= (len(strij)+1): # not minimum
                    continue
                queue.put((for_b, list(strij + [x])))  # match

            else: # miss-match
                current_miss = len(list(strij) + [x])   # update min length

                if size_min_absent > current_miss or size_min_absent == -1:
                    size_min_absent = current_miss
                if size_min_absent == len(list(strij) + [x]):
                    min_absent.append(list(strij) + [x])
    new_list = []
    for elem in min_absent:
        if len(elem) == size_min_absent:
            new_list.append(elem)
    return new_list
```

$O(|w|+1) = O(|w|)$ → optimal bzgl von Output alf

GIK

Representing Suffix Trees

→ ST(S$) represented by two tables T_{leaf} and T_{branch}

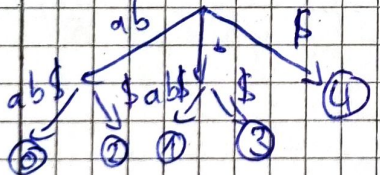
↳ ∀ leaf number j, $j \in [0, n]$, $T_{leaf}[j]$ stores a reference to the right brother of leaf $\overline{x_j}$, where x_j denotes the suffix of S$ begin at j.
If there is no such brother, then $T_{leaf}[j]$ is nil-reference

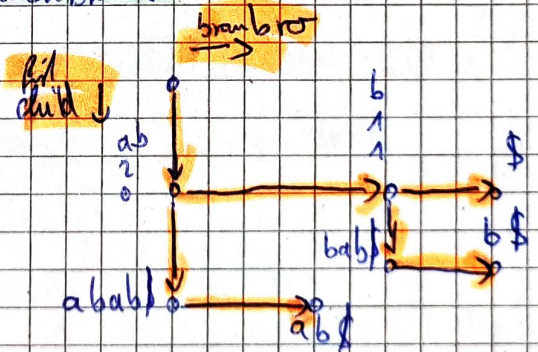
↳ ∀ branching node $\overline{w}$, $T_{branch}[\overline{w}]$ stores a branch record consisting of:

1) first-child refers to the first child of $\overline{w}$
2) branchbrother refers to the right brother of $\overline{w}$, or nil (if no bro)
3) depth is the depth of $\overline{w}$, depth $= |w|$
4) suffixpos is some $j \in [0, n]$, s.t. w is a prefix of x_j
5) suffixlink refers to the branching node $\overline{v}$, if w is of the form av for $a \in A$ and $v \in A^*$

successors of a branching node are found in a list of elements linked via first child and branchbrother

eg: ababa$





vertex without vertical edge → suffix

Representing Suffix Trees

→ Denote a record component: $\overline{w}.c$, $\nabla_{branch}[\overline{w}] = c$

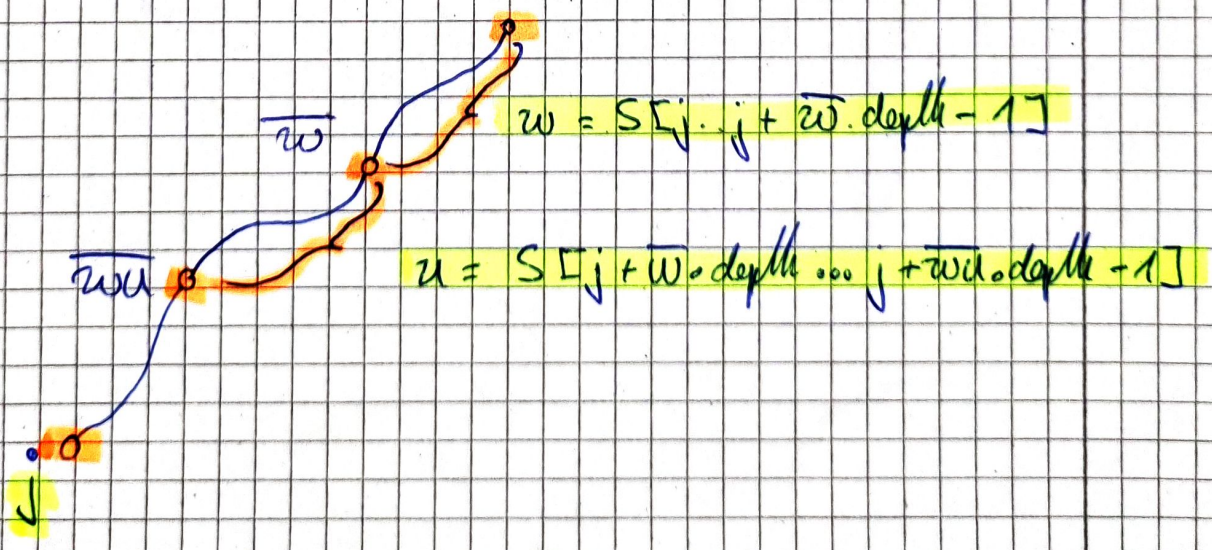
→ suffix position j of some branching node $\overline{wu}$ tells us that the leaf $\overline{x_j}$ occurs in the subtree below node $\overline{wu}$
↳ wu is prefix of x_j of length $\overline{wu}.depth$
$wu = S[j \,..\, j + \overline{wu}.depth - 1]$

→ if $\overline{w}$ is parent of $\overline{wu}$, $w = S[j \,..\, j + \overline{w}.depth - 1]$

⇒ the label of the incoming edge of node $\overline{wu}$ can be obtained by skipping the first $\overline{w}.depth$ characters of wu, where $\overline{w}$ is $\overline{wu}.parent$

Lemma 1:
If $\overline{w} \xrightarrow{u} \overline{wu}$ is an edge in $ST(S\$)$ and $\overline{wu}$ is a branching node, then we have $u = S[i \,..\, i+\ell-1]$, $i = \overline{wu}.suffixpos + \overline{w}.depth$ and $\ell = \overline{wu}.depth - \overline{w}.depth$

Lemma 2:
If $\overline{w} \xrightarrow{u} \overline{wu}$ is an edge in $ST(S\$)$ and $\overline{wu} = \overline{x_j}$ (leaf), $j \in [0, n]$, $u = S[i \,..\, n]\$$ where $i = j + \overline{w}.depth$



GST

Representing Suffix Trees

→ first child, sibling, and $T_{leaf}[j]$ are int's $\in [0, n]$

→ extra bit → refers to a leaf or branching node, T_{leaf} or T_{branch}

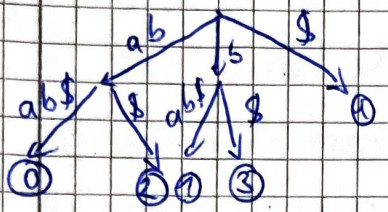
→ each leaf x_j is referred by leaf number j

→ q branching nodes in $ST(S\$)$

→ Let $b_0, \dots, b_{q-1}$ sequence of branching nodes.

→ b_i, i = node number, b_0 = root

eg: 0 1 2 3 4
a b a b $



T_{leaf}:

leaf	abab$	bab$	ab$	b$	$
leaf #j	0	1	2	3	4
$T_{leaf}[j]$	2	3	nil	nil	nil

T_{branch}:

branching node	root	ab	b
node #	0	1	2
first child	1b	0	1
sibling	nil	2b	4
depth	0	2	1
suffix pos	0	2	3
suffix link	/	2b	0b

↳ refers to branching node v
w = av

n integers for T_{leaf}

5q integers for T_{branch}

↳ Total $n + 5q$

$= O(n + 5q)$

Worst case $q = n$ → 6n space

but usually even less space

Gelb A 4

(4.5)

Implizit die Menge aller Suffixnummern für jeden Knoten

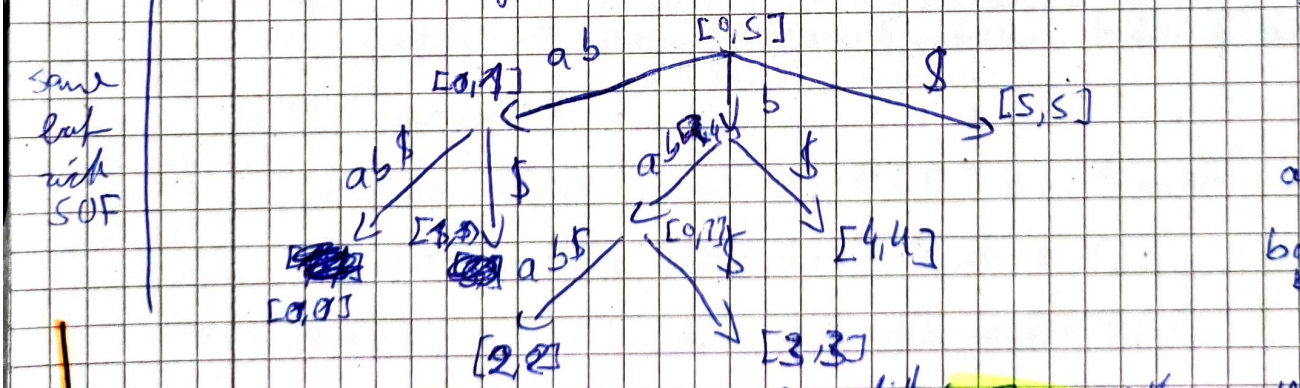
Speicherung $\hat{=}$ $\mathcal{O}(4)$ Speicher, $|S| = n$

Each node contains an interval $[a,b]$ (two numbers) (where a is equal to the smallest suffix number which can be reached from that node and b is the largest suffix number which can be reache from that node)

1) Construct Suffix Tree
2) DFS until leaf node
3) Interval for each node is initially empty
4) When you reach a leaf node, adjust the interval of its parent with the help of the suffix number SUF[j]
5) Backup and merge the intervalls of suffix numbers SUF[j] from the child nodes

$\mathcal{O}(4)$ Memory since number of branchj nodes $\mathcal{O}(n)$

for each branchj node 2 integers



babab$ 0
abab$ 1
bab$ 2
ab$ 3
b$ 4
$ 5

ababa$ 0
aba$ 1
babab$ 2
bab$ 3
b$ 4
$ 5

→ Save Intervall $[a,b]$ of node for which SUF[a] is the smallest suffix which can be reach from the node and SUF[b] is the largest suffix which can be reach from the node.

$\mathcal{O}(n)$ { SUF is such that $S_{SUF[0]} < \dots < S_{SUF[n]}$, sequence of suffix
S$ in ascending order

GIK

Enhanced Suffix Arrays

→ represent Suffix Trees

Lexicographic Order:

By $\prec$ we denote the lexicographic order on strings induced by the alphabet order $<$ on $A \cup \{\$\}$ for which we assume that \$ is the largest symbol.

For any $i, i \in [0, n]$, S_i denotes the suffix of $S\$$ starting at position i, $S_i = S[i \dots n-1]\$$

Enhanced Suffix Array

enhanced suffix array for S consists of tables SUF and LCP:

Permutation of $[0..n]$

- → SUF is a table of length $n+1$ indexed from 0 to n s.t. $S_{SUF[0]} \prec \dots \prec S_{SUF[n]}$, sequence of suffix of $S\$$ in ascending order ↑ lexicographic
- → LCP is a table of length n indexed 1 to n. For any $i \in [1, n]$, $LCP[i]$ is the length of the lcp of $S_{SUF[i-1]}$ & $S_{SUF[i]}$

→ suffix array is SUF alone

→ Idea: SUF defines the order of the leaves of $ST(S\$)$ and table LCP defines the shape of the tree by specifying the depth of branching nodes

Suffix-Intervals:

is a pair (ℓ, r), s.t. $0 \leq \ell \leq r \leq n$. It is written $[\ell, r]$ and refers to table SUF, denoting the suffixes $S_{SUF[i]}$ $\forall i \in [\ell, r]$.

Any $i \in [\ell, r]$ is a index in the suffix-interval $[\ell, r]$. $r - \ell + 1$ is width of $[\ell, r]$

Enhanced Suffix Array

ℓ-interval:

is a suffix interval $[\ell, r]$, s.t.

1) $0 \leq h \leq n+1$
2) $lcp[i] \geq h$, $\forall i \in [\ell+1, r]$
3) $\ell = 0$ or $lcp[\ell] < h$
4) $r = n$ or $lcp[r+1] < h$

-> h is depth of the ℓ-interval $[\ell, r]$ (r: boundaries)

-> all suffixes in an ℓ-interval have a common prefix w of length h

↳ all suffixes of $S\$$ with the same prefix w are contained in $[\ell, r]$

↳ $[\ell, r]$ is h-interval for w => h-$[\ell, r]$

-> h-intervals corresponds one-to-one to the location of the corresponding suffix tree.

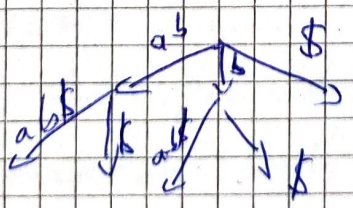
↳ subtree below the location represents the suffixes of $S\$$ of which w is a prefix -> start location leaf annotations $\in [\ell, r]$ of SUF

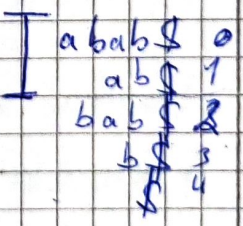
-> $h = |w|$, $0 \leq h \leq n+1$

=> lcp for all pairs of suffixes in $[\ell, r]$: -> is at least h

=> given ℓ-interval representing the substring w of $S\$$, one can uniquely derive the corresponding location of w

eg:





($\overline{b}$, a, b$, bab$) ~~~ 2-[3,4]

Gilli

Enhanced Suffix Array

→ ℓ-interval generalizes lcp-interval

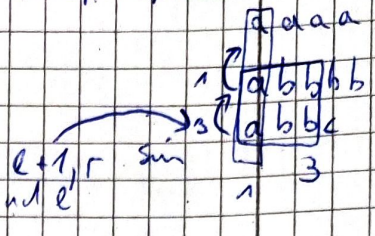
→ ℓ-interval is an lcp-interval $[l,r]$ with lcp-value ℓ, if

$l < r$ and $lcp[i] = \ell$, $\exists\, i \in [l+1, r]$

lcp-interval:

lcp-interval of lcp-value ℓ is a suffix interval $[l,r]$, s.t.

1) $l < r$
2) $lcp[i] \geq \ell$, $\forall\, i \in [l+1, r]$
3) $lcp[i] = \ell$, $\exists\, i \in [l+1, r]$
4) $l = 0$ or $lcp[l] < \ell$
5) $r = n$ or $lcp[r+1] < \ell$



→ one-to-one relation: lcp-interval ↝ interior node of ST

→ $\overline{w}$ branching node of string depth ℓ

→ l, r s.t. $\{ S_{SUF[i]} \mid l \leq i \leq r \}$ is the set of leaves in the subtree below $\overline{w}$ → $\overline{w}$ ↝ lcp-interval $[l,r]$ of lcp-value ℓ

From Suffix Tree to Suffix Array

depth-first traversal on ST(\$S\$) to generate SUF

leaves → SUF is filled from left to right

→ Each time a leaf $\overline{S_i}$ is visited, one stores index i in the next entry of table SUF

→ $\mathcal{O}(n)$ Runtime

branch → table LCP can also be filled in this traversal, because the depths of branching nodes are the lengths of the lcp of a pair of neighboring suffixes